

INF2003 Project Report G07

NAME	STUDENT ID
Borse Srushti Ganesh	2401618
Muhammad Adli Bin Jumahat	2401516
Subhashini Rajkumar	2402769
Lai Yueyin Shyann	2402456
Swathi Ravichandran	2402695
Heng Sheng Yao Dan	2400890

1. Application Description

Our application is a database system designed specifically for game developers. It centralizes and organizes game-related data, such as game sales, by collecting information from various popular gaming platforms.

Game developers often lack a single, reliable source of consolidated data when researching market trends or planning their next title. Relevant data like sales figures and platform-specific trends are usually fragmented across multiple platforms.

This database aims to collate and unify game-related data into a central repository, enabling developers to gain insights, analyze trends, and make more informed decisions during the game development process.

2. System Architecture

Our project is a database application using both relational and non-relational databases. The system's main components are as shown below:

- GUI: The front-end, handling user interaction such as login and CRUD operations. It sends user requests and receives processed data for display.
- Application Logic/ Backend Services receives GUI requests, manages data flow, and interacts with both databases to store and retrieve information about games and reviews.
- Relational Database (SQL): Stores structured data such as the different tables from the ER diagram (e.g., Game, Region, Achievement). It is accessed by Application Logic using SQL for CRUD and complex queries.
- Non-Relational Database (NoSQL): Stores less structured data, complementing the SQL databases, such as key-value or document data for game reviews and user metadata.

Appendix A contains the system architecture diagram.

3. Data

Datasets

Datasets used for SQL databases: [Game Profile](#), [Video Game Sales](#). [NoSQL](#) used the reviews dataset. We scaled all the data down to about 256 rows. SQL Database uses both The Video game sales dataset which contains information of games sales by region and The Gaming Profile dataset which contains information about games.

4. Functionalities and Project Management

The SQL side of the system manages all structured data related to games, companies, genres, languages, platforms, achievements, and sales. It supports CRUD operations for these entities, allowing admins to add, update, or delete games and their associated metadata.

The noSQL side manages reviews, as the reviews dataset is unstructured. This functionality allows developers to gauge player sentiment on existing games, which is useful when they create similar games or update current ones. To support the integrity of reviews, developers are not allowed to edit reviews on the webpage. Administrators can upload new reviews by CSV, parsed by game ID and delete reviews.

The system supports key functions like input validation, CRUD operations, complex queries for analytics, and transaction management. It handles bulk uploads, manages interactions between UI and databases, applies business logic, and ensures efficient access to both structured and unstructured game data.

Gantt chart, showing our project management, is in Appendix B.

5. Database Implementation

1. ER Diagram

The ERD is reflected in Appendix C.

2. List of Tables and Constraints

Main Tables: Represent the individual entity in the database

- Game, Platform, Company, Achievement, Language, Genre, Region

Join Tables: Represent the many-to-many relationship between main tables.

- Game Developer, Game Publisher, Supported Language, Genre in Game, Game on platform, Sales per Region

The summary table list and foreign key list are in appendix D. Relational Constraints list in appendix E.

3. Basic CRUD Functions

Only AdminUser have the privilege to Access to CRUD Functions, this is done by implementing a privilege field in our MongoDB User collection. Only Users with “admin” as their privilege field will be classified as an AdminUser. If a non AdminUser account tries to access CRUD Functions from the backend, the CRUD won’t go through. Example of this error message in Appendix F.

SQL

1. Create (Add)

a. Create Game

AdminUser can insert a new game entry through add game form or can insert multiple game entries through csv file uploads. The function will first check there is an existing game entry by comparing its name and list of developers and publishers. This is because different games can happen to have the same name but made by different companies. If there isn’t an existing game entry, the function will insert into the database along with the game entry’s related data (new game entry, new company entry if needed, new achievement entry if included, new entry for each join tables)

b. Create Game Sales

AdminUser will be able to add new game sales for the regions games are sold in. The function will check if the record for the game on platform exists, else it will add a new row into the table. It will then take the region sales values from the form, clean it and add it to the sales per region table, before updating the total copies sold in the game on platform table.

c. Create Platform

AdminUser can create a new gaming platform and link it to existing games by specifying the platform details and associating selected games with prices and release dates.

d. Create Game’s Achievements

Sql Query: Getting every game entry’s id, name and list of game developers and publishers to display info in add achievement form. Concatenate the list of developers and publishers into 1 string respectively. Code snippet in appendix G.

AdminUser can insert new achievements to existing games through add achievement form. AdminUser first needs to select which existing game to add achievement. Selecting an existing game will display its list of game developers and publishers so the AdminUser can ensure they are adding achievements to the correct game.

e. Create Language/Company/Genre

AdminUser will be able to create a new Language/Company/Genre into our database by doing an INSERT query , it will insert to the corresponding collection depending on the user choice through the route. Code snippet in appendix H.

2. Read

a. Index (main page)

This page lists all the names of games that the server holds information off. The page provides filter search by name, genre, language, and platforms. By selecting any games listed in the panel, a redirect will bring the user to its game details page.

b. Recently Viewed (NoSQL involved)

This page lists all the recently visited game detail pages for a user. Different users will see their own recently viewed page as the recently viewed information is retrieved from the user's documentation in the users collection of MongoDB.

c. Game Details

The Game Details page shows information of the selected game. From there, the following details of the game are genre, language, supporting platforms, developers, and publishers. This page also provides links to its Achievements and Reviews.

- i. Game Reviews (NoSQL involved) - display all reviews tied to the game
- ii. Game Achievements - display all achievements tied to the game

d. Game Sales Detail

The Game Sales Detail shows the breakdown of sales by platform and by region. The data is displayed from the game on platform table and the sales per region table respectively. There are also 3 charts which display sales data for easy visualisation and comparison. The 3 charts are sales by platform, sales by region and sales by platforms by region (for further breakdown and summary). Code snippet in Appendix I.

3. Update

a. Update Game Sales

AdminUser can update the game sales for regions here. They will be able to choose from the existing data in the table, to update the values of. The sales value will be updated in the sales per region table and the total copies sold will be updated in the game on platform table as well.

b. Update Company

AdminUser would be able to change pre-existing company names with an UPDATE query. If successful, a message prompt will inform the user of the update made. Code snippet in Appendix J.

c. Update Achievement

AdminUser can update existing achievements through add achievement form. In that function, selecting a game will list out existing achievements tied to that game. AdminUser can select any of the existing achievements to update the name or description of that achievement.

4. Delete

a. Delete Language/Company/Genre

AdminUser will be able to Delete Language/Company/Genre into our database via a DELETE WHERE query , the corresponding collection will be chosen based on user choice through the route. Code snippet in Appendix K

b. Delete Game

AdminUser is able to delete an existing game. This will delete the game from the game table and all its associated child table records. The SQL has been setup to delete records from the child tables to DELETE ON CASCADE.

c. Delete Achievement

AdminUser can update existing achievements through add achievement form. In that function, selecting a game will list out existing achievements tied to that game. AdminUser can select any of the existing achievements to delete.

NoSQL

1. **Create:** add reviews

- Ensures CSV includes “reviews” and “newgameID” columns. Reviews converted to strings and mapped to whitespace to ensure fields with no reviews are skipped. Rows where “newgameID” contain non-integer values are also skipped. Rows are only added to the DB after these parameters are fulfilled.
- The SELECT query retrieves game names for all game IDs that have been uploaded.
- row.mapping transforms rows into keyvalue python dictionary.
- “Summary append” creates a list of dictionaries and passes them to the HTML template to display all added reviews.

2. **Read**

- Games are selected based on their GAMEID and displayed

3. **Update**

- Users are stored in NOSQL, their recently visited games are updated.
- Finds regular users within the database using find_one. If game has been viewed before, add it to the front of the list. Updates the list using update_one.

4. **Delete**

- The MongoDB query filters documents where the _id field is in the list of ObjectIds ({"_id": {"\$in": obj_ids}}). The delete_many method removes all relevant documents in the gamingreviews collection.

4. Advanced Functions

- Index by newgameID to speed up read and delete queries for reviews
- Backend security to prevent anonymous, unauthorised users from accessing CRUD functions

- CSV multi upload for games and reviews which checks through CSVs to validate entries before adding information to respective webpages.

5. Constraints with Justification

Limitations	Issues
Architecture & Integration • Dual-database setup	• Lacks cross-data transactions • Manual consistency required
Security and Scalability	• Designed for local server setup ◦ Large-scale, real-time performance untested • Basic privilege control via MongoDB privilege `field` ◦ Not production-secured ◦ No audit trail for Admin CRUD operations
Data Update Frequency	• Real-time updates are not supported

6. System-Level Database Integration

Flask: Python web framework to build the server and API endpoints.

Javascript: Scripts for dynamic data display and forms

PyMongo: Used to connect and interact with MongoDB.

MongoDB: NoSQL database used to store game reviews.

The unstructured nature of the source data, which contains user-generated text with inconsistent formats, variable lengths, and missing fields, is suitable for NOSQL analysis. Its document model allows reviews to be stored without requiring a rigid schema, making it easier to handle and clean messy or incomplete data.

MySQL: Relational database to store the rest of the supporting data tables.

Its structured schema and support for relationships (via foreign keys) make it suitable for managing normalized data with defined constraints.

SQLAlchemy: To connect and interact with MySQL

Performance Metrics

The index speeds gameId search. Upon using this snippet, "result = mongo.db.gamingreviews.find({"newgameid": 6}).explain() print(result)", the stats revealed that indexing helped perform a targeted search rather than the entire collection. Only 5 documents were scanned, with a 0 millisecond execution time.

Without the index, 2496 documents were searched with a 2 millisecond execution time.

7. Discussion and Reflection

Heng Sheng Yao Dan 2400890

One way to further improve our project would be code organization, we didn't expect our codes to grow this huge when we started out on the project. Debugging was quite tiresome during the end of our project due to the long lines of code we have, but we managed to debug it successfully due to everyone's hardwork.

Borse Srushti Ganesh 2401618

This project helped me apply the concepts learnt in the DBMS mod, while also allowing me to explore advanced functionalities. I was able to learn what type of datasets are suitable for SQL and noSQL databases. Working more extensively on the NoSQL side was initially daunting, as it required a different way of thinking, especially when ensuring data consistency without traditional constraints like foreign keys. However, it gave me a better appreciation for schema flexibility and scalability in real-world applications.

If I were to re-do the project, I will choose a NOSQL feature that can be better integrated with the SQL side, so I can get more hands on experience on SQL as well. The current NOSQL dataset was separated completely from the SQL side, which, while advantageous for web development logic and in-depth experience, did not allow me to fully immerse myself in SQL. I have, however, learnt from my groupmate's talented SQL codes and module content.

Subhashini Rajkumar 2402769

During this project, I focused primarily on the SQL side, where I implemented features for creating new gaming platforms and linking them to existing games. This involved designing and managing relational tables, as well as writing and optimizing SQL queries for data handling. I also contributed to the MongoDB side by working on data cleaning tasks, ensuring that the review data was properly formatted and integrated. This experience strengthened my skills in both relational and non-relational database management and gave me practical insights into handling real-world data consistency and integration challenges.

Swathi Ravichandran 2402695

This project taught me a lot about building websites and managing databases, especially with a normal setup. Putting our sample data into the database was challenging, needing much cleaning and checking to ensure correctness. This showed me how much work goes into getting data ready for a system. Team discussions were also key for deciding on our database design and how to structure features, helping us build a working system. My SQL skills greatly improved. I learned to build various SQL queries, from simple ones to more complex ones, to get the exact data needed. This helped me understand SQL and web development much better. While happy with this, I do wish I had spent more time learning about NoSQL databases for a wider view of data storage options.

Lai Yueyin Shyann 2402456

In this project, I learned that databases are a fascinating topic in practice. Working primarily on the SQL side, handling and cleaning data, and ensuring its relevance and consistency, made me realize how much effort is required even before working with the actual database. I gained many insights just by reviewing my teammates' code, and our frequent discussions helped us make steady progress. Overall, I enjoyed this experience, which taught me a lot and strengthened my skills in database management.

Muhammad Adli Bin Jumahat 2401516

This project was pretty challenging in a sense of application design and development itself. When starting the project, we had tons of different ideas but struggled to find a good live dataset to compliment the idea. So we had to find a good dataset and come up with an idea of what we can do with the dataset instead. That itself leads us to another set of issues where during the actual development of the application, we had to drop certain features from our applications as we weren't making good use of some part of the data.

We first started out working on how to integrate the kaggle datasets into sql for game and player data. We couldn't simply import the csvs directly as there were too many entries names that were not standardized such as the different names of the same company, genre and even game name across the different platforms in the dataset. So we went ahead and decided to import manually so we can standardize the entries ourselves.

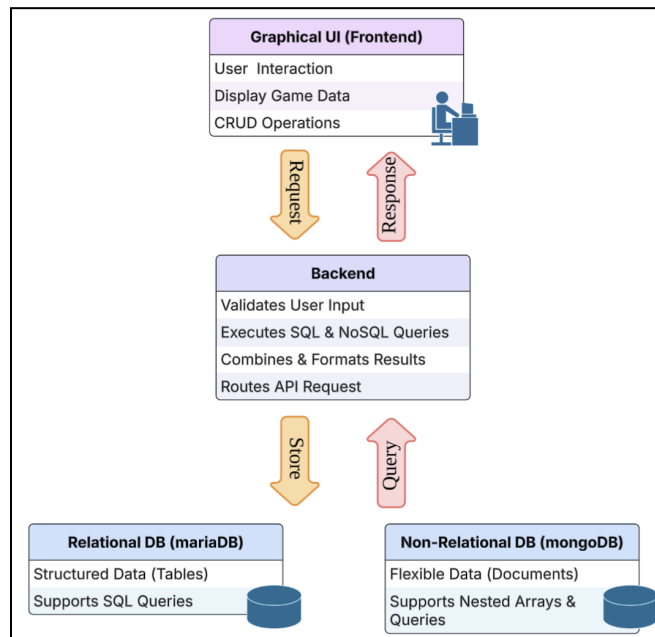
As our goal then was to simply import the entries from the different csvs to sql, we only had come up with a general application idea of what we can do with the data. This led us to integrate the player data where we originally had Player Table as main table, and Games Owned, Achievements Owned and Player on Platform as join tables between the Player table and other main tables. Only when we are trying to develop the application features directly, did we realize that we do not have any good feature to use with the player data where some of the tables did not even end up being used. Thus, we decided to simply drop the player idea and player tables.

This is a lesson learnt that we should not be too eager to start development and that having a detailed application design and specifications before development is very crucial step so that we can prevent integrating data or features that we would end up removing as we would have foresaw the lack of use for player data during the application design stage.

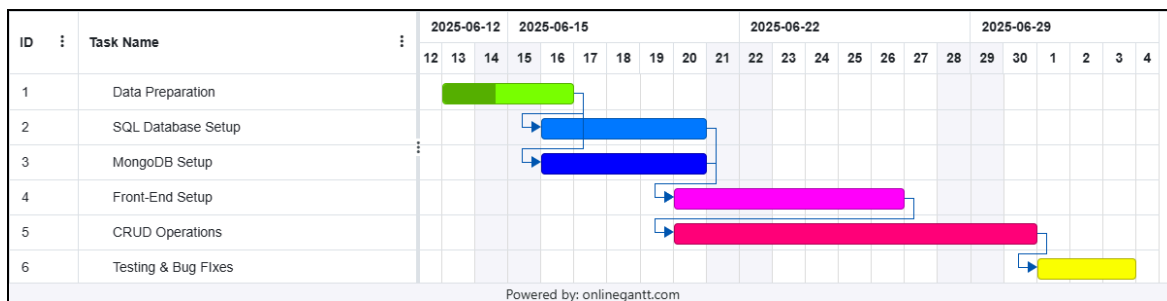
Overall, the project was a fun and fruitful experience and I definitely learned a lot throughout the different stages in this project.

8. Appendices

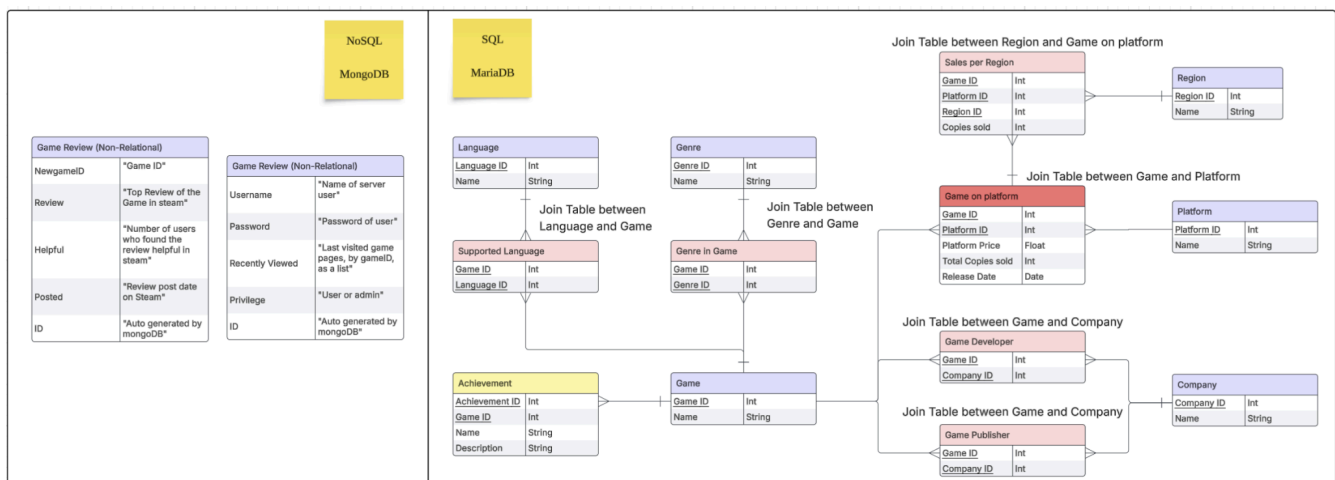
Appendix A: System Architecture



Appendix B: Gantt Chart



Appendix C: ERD Diagram



Appendix D: Table Names

Table Name

Primary Keys

Foreign Keys

Language	Language_ID	—
Genre	Genre_ID	—
Region	Region_ID	—
Company	Company_ID	—
Platform	Platform_ID	—
Game	Game_ID	—
Achievement	Achievement_ID	Game_ID → Game
Supported Language	Game_ID, Language_ID	Game_ID → Game, Language_ID → Language
Genre in Game	Game_ID, Genre_ID	Game_ID → Game, Genre_ID → Genre
Game on Platform	Game_ID, Platform_ID	Game_ID → Game, Platform_ID → Platform
Game Developer	Game_ID, Company_ID	Game_ID → Game, Company_ID → Company
Game Publisher	Game_ID, Company_ID	Game_ID → Game, Company_ID → Company
Sales per Region	Game_ID, Platform_ID, Region_ID	Game_ID → Game, Platform_ID → Platform, Region_ID → Region
Game Review (NoSQL)	ID (auto)	
User Review (NoSQL)	ID (auto)	

Appendix E: relational constraints

One-to-Many Relationships

- **Game → Achievement**
- **Language → Supported Language**
- **Genre → Genre in Game**
- **Company → Game Developer, Game Publisher**
- **Platform → Game on Platform**
- **Region → Sales per Region**

Many-to-Many Relationships (via join tables)

- **Game ↔ Language** (via Supported Language)
- **Game ↔ Genre** (via Genre in Game)
- **Game ↔ Platform** (via Game on Platform)
- **Game ↔ Company** (via Game Developer, Game Publisher)
- **(Game + Platform) ↔ Region** (via Sales per Region)

Appendix F: Security Function

```
C:\Users\danhe>curl -X POST http://localhost:5000/delete/language ^
More? -H "Content-Type: application/x-www-form-urlencoded" ^
More? -d "languageName=Japanese"
{"error": "Access denied: only Admin account can access this page."}
C:\Users\danhe>
```

Appendix G: Create Game Achievement Code Snippet

```
with games_engine.connect() as conn:
    result = conn.execute(text("""
        SELECT
            g.gameID,
            g.gameName,
            GROUP_CONCAT(DISTINCT cdev.companyName SEPARATOR ' || ') AS gameDeveloper,
            GROUP_CONCAT(DISTINCT cpub.companyName SEPARATOR ' || ') AS gamePublisher
        FROM Game g
        LEFT JOIN GameDeveloper gd ON g.gameID = gd.gameID
        LEFT JOIN Company cdev ON gd.companyID = cdev.companyID
        LEFT JOIN GamePublisher gp ON g.gameID = gp.gameID
        LEFT JOIN Company cpub ON gp.companyID = cpub.companyID
        GROUP BY g.gameID, g.gameName""")).fetchall()

    db_game_info = {}
    for row in result:
        db_game_info[row[0]] = {"gamename":row[1],"gamedeveloper":row[2] or "No developers listed","gamepublisher":row[3] or "No publishers listed"}

    return render_template('addAchievement_form.html', db_game_info=db_game_info, Admin_User=Admin_User)
```

Appendix H: create Language, Company, Genre Code Snippet

```
@app.route('/create/company', methods=['POST'])
def create_company():
    if 'username' not in session or session['username'] not in Admin_User:
        if request.accept_mimetypes['application/json'] >= request.accept_mimetypes['text/html']:
            return {"error": "Access denied: only Admin account can access this page."}, 403
        else:
            return render_template('index.html', error="Access denied: only Admin account can access this page.")

    name = request.form['companyName']
    with games_engine.begin() as conn:
        conn.execute(text("INSERT INTO Company (companyName) VALUES (:name)"), {"name": name})

    return render_template('create.html', success=f"Company '{name}' added successfully!", Admin_User=Admin_User)
```

Appendix I: Game Sales Detail Code Snippet

```
raw_platform_region_sales_for_chart = conn.execute(text("""
SELECT
    P.platformName,
    COALESCE(SUM(CASE WHEN R.regionName = 'NA' THEN SPR.Copies_sold ELSE 0 END), 0) AS NA,
    COALESCE(SUM(CASE WHEN R.regionName = 'EU' THEN SPR.Copies_sold ELSE 0 END), 0) AS EU,
    COALESCE(SUM(CASE WHEN R.regionName = 'JP' THEN SPR.Copies_sold ELSE 0 END), 0) AS JP,
    COALESCE(SUM(CASE WHEN R.regionName = 'Other' THEN SPR.Copies_sold ELSE 0 END), 0) AS Other
FROM Platform AS P
LEFT JOIN GameOnPlatform AS GOP ON P.platformID = GOP.platformID
LEFT JOIN SalesperRegion AS SPR ON GOP.gameID = SPR.GameID AND GOP.platformID = SPR.PlatformID
LEFT JOIN Region AS R ON SPR.RegionID = R.RegionID
GROUP BY P.platformName
ORDER BY P.platformName; """)).all()
```

Appendix J: Update Company Code Snippet

```
@app.route('/update_company', methods=['GET', 'POST'])
def update_company():
    if 'username' not in session or session['username'] not in Admin_User:
        return redirect(url_for('login'))
    with games_engine.connect() as conn:
        result = conn.execute(
            text("SELECT companyID, companyName FROM company")
        ).mappings().all()

    if request.method == 'POST':
        company_id = request.form['company_id']
        new_name = request.form['new_name']
        with games_engine.begin() as conn:
            # Update name
            conn.execute(
                text("UPDATE Company SET companyName = :new_name WHERE companyID = :company_id"),
                {"new_name": new_name, "company_id": company_id}
            )
        flash(f"Company name successfully changed to '{new_name}'.", "success")
        return render_template("update_company.html", companies=result, Admin_User=Admin_User)

    return render_template("update_company.html", companies=result, Admin_User=Admin_User)
```

Delete Language, Company, Genre

```

@app.route('/delete/company', methods=['POST'])
def delete_company():
    if 'username' not in session or session['username'] not in Admin_User:
        if request.accept_mimetypes['application/json'] >= request.accept_mimetypes['text/html']:
            return {"error": "Access denied: only Admin account can access this page."}, 403
        else:
            return render_template('index.html', error="Access denied: only Admin account can access this page.", Admin_User=Admin_User)
    name = request.form['companyName']
    with games_engine.begin() as conn:
        result = conn.execute(text("DELETE FROM Company WHERE companyName = :name"), {"name": name})

    if result.rowcount == 0:
        return render_template('delete.html', success=f"No company named '{name}' found to delete.", Admin_User=Admin_User)
    else:
        return render_template('delete.html', success=f"Company '{name}' deleted successfully!", Admin_User=Admin_User)

```